

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN – ĐIỆN TỬ



BÁO CÁO THỰC TẬP AI

BÀI 7: VISION TRANSFORMER

MÔN HỌC: THỰC HÀNH HỌC MÁY VÀ TRÍ TUỆ NHÂN TẠO
GIẢNG VIÊN HƯỚNG DẪN: PGS_TS. TRƯƠNG NGỌC SƠN
SINH VIÊN THỰC HIỆN: TRƯƠNG PHÚC THỊNH - 23161336

Tp. Hồ Chí Minh, tháng 6 năm 2026

MỤC LỤC

1. GIỚI THIỆU	1
1.1 Tổng quan về Vision Transformer.....	1
1.2 Bài toán phân loại ảnh CIFAR.....	1
1.3 Mục tiêu bài thực hành	1
2. CƠ SỞ LÝ THUYẾT	3
2.1 Tổng quan về Vision Transformer và dữ liệu đầu vào	3
2.2 Chia ảnh thành patch và xây dựng patch embedding	3
2.3 CLS token và positional embedding.....	4
2.4 Self-Attention và Multi-Head Self-Attention	5
2.5 Transformer Encoder, LayerNorm và MLP	6
2.6 Classification head, Softmax và Cross-Entropy Loss	6
2.7 Transfer learning, fine-tuning và tối ưu AdamW	7
2.8 Chỉ số đánh giá và khả năng tổng quát	8
2.9 So sánh CNN với ViT và ảnh hưởng của siêu tham số	8
3. QUY TRÌNH THỰC HIỆN.....	10
3.1 Môi trường và dữ liệu	10
3.2 Tiền xử lý và DataLoader	10
3.3 Xây dựng ViT-B/16 và CNN đối chứng.....	11
3.4 Pipeline huấn luyện và đánh giá	11
3.5 Thiết kế các thí nghiệm.....	11
4. KẾT QUẢ THỰC NGHIỆM.....	12
4.1 Chuẩn bị thư viện, môi trường và dữ liệu.....	12
4.1 Bài 2: Tải dataset CIFAR-10	14
4.2 Bài 2: Hiển thị ảnh trong dataset	15
4.3 Bài 3: Huấn luyện Vision Transformer trong 5 epoch	16

4.4 Bài 4: Tính accuracy và inference trên tập test	19
4.5 Bài 5: Thay đổi learning rate	20
4.6 Bài 6: Thử các batch size 16, 32, 64.....	21
4.7 Bài 7: So sánh Vision Transformer và CNN	24
4.8 Bài 8: Thử nghiệm dataset CIFAR-100.....	25
4.9 Bài 9: Thử mô hình ViT-B/32	26
4.10 Bài 10: Phân tích các trường hợp dự đoán sai	28
5. PHÂN TÍCH VÀ THẢO LUẬN	33
5.1 Đánh giá ViT-B/16 trên CIFAR-10.....	33
5.2 Ảnh hưởng của learning rate và batch size.....	33
5.3 So sánh ViT và CNN	33
5.4 CIFAR-100 và ảnh hưởng số lớp.....	33
5.5 So sánh patch size 16 và 32	33
5.6 Phân tích lỗi dự đoán	34
5.7 Lỗi thực nghiệm, hạn chế và hướng cải thiện.....	34
6. KẾT LUẬN.....	36

DANH MỤC HÌNH ẢNH

Hình 1: Kết quả hiển thị kiểm tra GPU	13
Hình 2: Kiểm tra dữ liệu CIFAR-10 và CIFAR_100	14
Hình 3: Kiểm tra mẫu dataset CIFAR- 10	15
Hình 4: Mười hai ảnh mẫu trong CIFAR-10 từ notebook.....	16
Hình 5: Kết quả huấn luyện Vision Transformer trong 5 epoch	17
Hình 6: Đường cong loss của ViT-B/16 trên CIFAR-10.....	18
Hình 7: Đường cong accuracy của ViT-B/16 trên CIFAR-10	18
Hình 7: Kết quả inference một ảnh automobile.....	19
Hình 8: Kết quả thay đổi learning rate.....	20
Hình 9: So sánh accuracy theo learning rate	21
Hình 10: So sánh accuracy theo các batchsize 16, 32, 64	22
Hình 11: So sánh accuracy theo batch size.....	23
Hình 12: So sánh thời gian huấn luyện theo batch size	23
Hình 13: Kết quả so sánh Vision Transformer và CNN cơ bản	24
Hình 14: Kết quả thử nghiệm dataset CIFAR-100	26
Hình 15: Kết quả thử nghiệm với mô hình VIT-B/32	27
Hình 16: Các trường hợp dự đoán sai.....	29
Hình 17: Mười hai trường hợp ViT-B/16 dự đoán sai.....	31
Hình 18: Tổng hợp các kết quả xử dụng trong mô hình.....	31

DANH MỤC BẢNG

Bảng 1: Thống kê dataset CIFAR-10	1
Bảng 2: So sánh cách tạo token của ViT-B/16 và ViT-B/32.....	4
Bảng 3: Môi trường và cấu hình thực nghiệm.....	10
Bảng 4: Siêu tham số mô hình ViT-B/16 chính	11
Bảng 5: Kết quả huấn luyện ViT-B/16 theo từng epoch	17
Bảng 6: Kết quả thử nghiệm learning rate.....	21
Bảng 7: Kết quả thử nghiệm batch size	22
Bảng 8: So sánh ViT-B/16 và Simple CNN	25
Bảng 9: Kết quả ViT-B/16 trên CIFAR-100	26
Bảng 10: So sánh ViT-B/16 và ViT-B/32	28
Bảng 11: Một số cặp nhãn thật - nhãn dự đoán sai.....	29
Bảng 12: Tổng hợp kết quả chính của Lab 7.....	31
Bảng 13: Lỗi/cảnh báo thực nghiệm và hướng khắc phục	34

1. GIỚI THIỆU

1.1 Tổng quan về Vision Transformer

Vision Transformer (ViT) là kiến trúc thị giác máy tính áp dụng Transformer Encoder trực tiếp lên chuỗi các vùng ảnh. Thay vì dùng convolution để quét đặc trưng cục bộ như CNN, ViT chia ảnh thành các patch có kích thước cố định, ánh xạ mỗi patch thành một vector embedding rồi dùng self-attention để học quan hệ giữa mọi vùng ảnh.

Bài thực hành sử dụng các mô hình vit_b_16 và vit_b_32 của torchvision. Trọng số pretrained của ViT-B/16 được tải từ dữ liệu cục bộ trên Kaggle; classification head được thay thành 10 lớp cho CIFAR-10 hoặc 100 lớp cho CIFAR-100. Kết quả, mã nguồn và hình minh họa trong báo cáo được lấy trực tiếp từ notebook bai-7.ipynb.

1.2 Bài toán phân loại ảnh CIFAR

CIFAR-10 gồm 60.000 ảnh RGB kích thước 32x32 thuộc 10 lớp: airplane, automobile, bird, cat, deer, dog, frog, horse, ship và truck. Tập train có 50.000 ảnh, tập test có 10.000 ảnh. CIFAR-100 có cùng số ảnh nhưng chia thành 100 lớp, vì vậy số mẫu mỗi lớp ít hơn và bài toán khó hơn.

Bảng 1: Thống kê dataset CIFAR-10

Tập	Số ảnh	Kích thước gốc	Số lớp	Vai trò
Train	50.000	32x32 RGB	10	Huấn luyện và augmentation
Test	10.000	32x32 RGB	10	Đánh giá mô hình

1.3 Mục tiêu bài thực hành

- Hiểu pipeline Image - Patch - Embedding - Transformer Encoder - Classification Head.
- Tải, tiền xử lý và trực quan hóa CIFAR-10 bằng torchvision và matplotlib.
- Fine-tune ViT-B/16 pretrained trong 5 epoch và đánh giá accuracy trên tập test.
- Khảo sát ảnh hưởng của learning rate và batch size đến accuracy và thời gian huấn luyện.

- So sánh Vision Transformer với Simple CNN, CIFAR-10 với CIFAR-100 và ViT-B/16 với ViT-B/32.
- Phân tích các ảnh dự đoán sai và đề xuất hướng cải thiện mô hình.

2. CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về Vision Transformer và dữ liệu đầu vào

Vision Transformer (ViT) là kiến trúc áp dụng cơ chế Transformer cho bài toán thị giác máy tính. Thay vì sử dụng các bộ lọc tích chập để quét trên ảnh như CNN, ViT chia ảnh thành nhiều vùng nhỏ gọi là patch, biến mỗi patch thành một token rồi xử lý chuỗi token bằng các khối Transformer Encoder. Nhờ cơ chế self-attention, mỗi patch có thể trao đổi thông tin trực tiếp với mọi patch khác, từ đó mô hình học được cả đặc trưng cục bộ và quan hệ toàn cục của ảnh.

Trong bài thực hành, dữ liệu chính là CIFAR-10 gồm ảnh màu RGB kích thước 32×32 và 10 lớp. Mô hình ViT-B/16 của torchvision nhận ảnh 224×224 nên ảnh CIFAR phải được resize trước khi đưa vào mạng. Việc phóng lớn chỉ nội suy giá trị pixel chứ không tạo thêm chi tiết mới; vì vậy chất lượng ảnh gốc vẫn ảnh hưởng đáng kể đến khả năng phân loại. Sau ToTensor(), giá trị pixel được đưa về $[0,1]$ và chuẩn hóa theo từng kênh màu:

$$X_{\text{norm}}^{(c)}(x, y) = \frac{X^{(c)}(x, y) - \mu_c}{\sigma_c}$$

Trong đó $X^{(c)}(x, y)$ là giá trị pixel tại vị trí (x, y) của kênh c ; μ_c và σ_c lần lượt là trung bình và độ lệch chuẩn của kênh đó. Notebook sử dụng mean $[0,485; 0,456; 0,406]$ và standard deviation $[0,229; 0,224; 0,225]$ của ImageNet để đầu vào phù hợp với phân phối dữ liệu đã dùng khi pretrain ViT. Tập train còn được tăng cường bằng RandomHorizontalFlip và RandomCrop nhằm tạo thêm biến thể của ảnh, giảm overfitting mà không làm thay đổi nhãn.

2.2 Chia ảnh thành patch và xây dựng patch embedding

ViT không xử lý từng pixel riêng lẻ mà chia ảnh kích thước $H \times W$ thành các patch không chồng lấp kích thước $P \times P$. Số patch tạo ra được xác định bởi:

$$N = \frac{H}{P} \cdot \frac{W}{P} = \frac{HW}{P^2}$$

Mỗi patch có P^2 pixel trên C kênh màu. Patch thứ i được trải phẳng thành một vector như sau:

$$p_i = \text{Flatten}(X_i) \in \mathbb{R}^{P^2 C}, i = 1, 2, \dots, N$$

Vector patch ban đầu có P^2C phần tử, trong khi Transformer làm việc trong không gian embedding D chiều. Do đó, mỗi vector patch được chiếu tuyến tính qua ma trận E và cộng bias b để tạo patch embedding:

$$e_i = p_i E + b, e_i \in \mathbb{R}^D$$

Với ảnh 224×224 , ViT-B/16 sử dụng $P=16$ nên tạo $14 \times 14 = 196$ patch; ViT-B/32 sử dụng $P=32$ nên chỉ tạo $7 \times 7 = 49$ patch. Patch nhỏ giữ lại nhiều chi tiết hơn nhưng làm chuỗi token dài và tăng chi phí attention. Patch lớn giảm thời gian và bộ nhớ, đôi lại có thể làm mất các đặc trưng nhỏ vốn đã khó quan sát trong ảnh CIFAR-10 sau khi resize.

Bảng 2: So sánh cách tạo token của ViT-B/16 và ViT-B/32

Mô hình	Patch size	Số patch	Số token	Nhận xét
ViT-B/16	16×16	196	197	Giữ nhiều chi tiết hơn, chi phí attention cao hơn
ViT-B/32	32×32	49	50	Chuỗi ngắn hơn nhưng mất nhiều thông tin cục bộ

2.3 CLS token và positional embedding

Sau khi tạo patch embedding, ViT thêm một vector học được gọi là CLS token vào đầu chuỗi. CLS token không đại diện cho một vùng ảnh cụ thể mà đóng vai trò thu nhận và tổng hợp thông tin từ toàn bộ patch qua các lớp self-attention. Biểu diễn cuối cùng của token này được dùng cho bài toán phân loại ảnh.

Self-attention tự nó không phân biệt thứ tự hoặc vị trí không gian của các patch. Vì vậy, positional embedding được cộng vào chuỗi đầu vào để mô hình nhận biết patch nằm ở bên trái, bên phải, phía trên hay phía dưới ảnh. Chuỗi token đầu vào của Transformer được biểu diễn:

$$Z_0 = [x_{\text{cls}}; e_1; e_2; \dots; e_N] + E_{\text{pos}}, Z_0 \in \mathbb{R}^{(N+1) \times D}$$

Trong đó x_{cls} là CLS token, e_i là embedding của patch thứ i và E_{pos} là positional embedding học được. Với ViT-B/16, chuỗi có 197 token gồm 196 patch

token và một CLS token. Thành phần vị trí giúp mô hình duy trì bố cục ảnh sau khi ảnh đã được chuyển thành chuỗi.

2.4 Self-Attention và Multi-Head Self-Attention

Self-Attention là thành phần quan trọng nhất của Transformer Encoder. Từ ma trận đầu vào X , mô hình tạo ba biểu diễn Query (Q), Key (K) và Value (V) thông qua các phép chiếu tuyến tính có trọng số học được:

$$Q = XW_Q, K = XW_K, V = XW_V$$

Query thể hiện thông tin mà một token đang tìm kiếm, Key mô tả đặc trưng dùng để so khớp, còn Value chứa nội dung sẽ được tổng hợp. Độ tương đồng giữa Query và Key được tính bằng tích vô hướng, chia cho căn bậc hai của d_k để tránh logits quá lớn trước Softmax:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

$$\text{Attention}(Q, K, V) = AV = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Ma trận A chứa trọng số chú ý giữa mọi cặp token. Giá trị A_{ij} càng lớn nghĩa là token i sử dụng nhiều thông tin từ token j . Cơ chế này cho phép một patch liên hệ trực tiếp với các vùng ở xa trong ảnh, khác với convolution thường phải đi qua nhiều lớp mới mở rộng được receptive field.

Để mô hình học nhiều kiểu quan hệ đồng thời, ViT sử dụng Multi-Head Self-Attention. Mỗi head có bộ ma trận chiếu riêng và tạo ra một biểu diễn attention độc lập:

$$\text{head}_r = \text{Attention}(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)}), r = 1, \dots, h$$

$$\text{MSA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

Các head có thể tập trung vào màu sắc, kết cấu, hình dạng hoặc quan hệ giữa các bộ phận của đối tượng. Kết quả từ h head được nối lại và chiếu qua W_O để trở về embedding dimension D . Tuy nhiên, attention phải lưu ma trận kích thước $N \times N$ nên chi phí tính toán tăng gần theo $O(N^2D)$; đây là lý do patch size ảnh hưởng mạnh đến tốc độ và bộ nhớ.

2.5 Transformer Encoder, LayerNorm và MLP

Một Transformer Encoder block gồm hai nhánh chính: Multi-Head Self-Attention và mạng MLP. ViT sử dụng cấu trúc Pre-Norm, tức LayerNorm được áp dụng trước mỗi nhánh. Đầu ra của từng nhánh được cộng với đầu vào bằng residual connection, giúp giữ thông tin gốc và làm gradient truyền ổn định qua mạng sâu:

$$U_l = Z_{l-1} + \text{MSA}(\text{LN}(Z_{l-1}))$$

$$Z_l = U_l + \text{MLP}(\text{LN}(U_l)), l = 1, 2, \dots, L$$

LayerNorm chuẩn hóa các phần tử trong từng token theo chiều embedding. Khác với BatchNorm phụ thuộc thống kê của batch, LayerNorm hoạt động ổn định ngay cả khi batch size thay đổi:

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

Trong công thức trên, μ và σ^2 là trung bình và phương sai của vector token; γ và β là hai tham số học được. Sau attention, từng token đi qua MLP gồm hai lớp tuyến tính và hàm kích hoạt GELU:

$$\text{MLP}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2$$

$$\text{GELU}(x) = x\Phi(x) \approx \frac{x}{2} \left[1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right]$$

Attention có nhiệm vụ trao đổi thông tin giữa các token, còn MLP biến đổi đặc trưng bên trong từng token. GELU làm giảm mềm các giá trị âm thay vì cắt hoàn toàn như ReLU, nhờ đó tạo chuyển tiếp trơn hơn và thường được dùng trong các kiến trúc Transformer.

2.6 Classification head, Softmax và Cross-Entropy Loss

Sau L encoder block, biểu diễn của CLS token được đưa qua classification head là một lớp tuyến tính. Với C lớp, head tạo vector logits $\mathbf{o}$ có C phần tử:

$$\mathbf{o} = W_{\text{cls}} Z_{\text{cls}} + b_{\text{cls}}$$

Logits chưa phải xác suất. Hàm Softmax chuyển chúng thành phân phối xác suất và chọn lớp có giá trị lớn nhất làm nhãn dự đoán:

$$p_c = \frac{\exp(o_c)}{\sum_{j=1}^C \exp(o_j)}, \hat{y} = \arg \max_c p_c$$

Trong notebook, C=10 với CIFAR-10 và C=100 với CIFAR-100. Khi chuyển bộ dữ liệu, classification head phải được thay để số đầu ra phù hợp, trong khi phần backbone có thể tiếp tục sử dụng trọng số pretrained. Mô hình được huấn luyện bằng Cross-Entropy Loss:

$$\mathcal{L}_{CE} = -\frac{1}{B} \sum_{i=1}^B \sum_{c=1}^C y_{ic} \log(p_{ic})$$

Với batch gồm B mẫu và C lớp, y_{ic} là nhãn thật dạng one-hot, còn p_{ic} là xác suất dự đoán. Loss nhỏ khi mô hình gán xác suất cao cho lớp đúng. Trong PyTorch, `nn.CrossEntropyLoss` nhận trực tiếp logits và kết hợp LogSoftmax với Negative Log-Likelihood để tăng độ ổn định số học.

2.7 Transfer learning, fine-tuning và tối ưu AdamW

Transfer learning là quá trình tái sử dụng mô hình đã học trên tập dữ liệu lớn cho bài toán mới. Trong notebook, kiến trúc ViT-B/16 được tạo bằng torchvision, sau đó nạp state_dict pretrained từ Kaggle Input và thay classification head cho CIFAR. Cách làm này giúp mô hình tận dụng các đặc trưng hình ảnh đã học từ trước thay vì khởi tạo toàn bộ trọng số ngẫu nhiên.

Có hai chiến lược thường dùng. Feature extraction đóng băng backbone và chỉ huấn luyện head, phù hợp khi dữ liệu hoặc tài nguyên hạn chế. Full fine-tuning cập nhật toàn bộ mạng, cho phép mô hình thích nghi tốt hơn với CIFAR nhưng cần learning rate nhỏ để tránh phá vỡ trọng số pretrained. Bài thực hành sử dụng full fine-tuning với `freeze_backbone=False`.

Optimizer AdamW duy trì moment bậc nhất m_t và moment bậc hai v_t của gradient g_t :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Do hai moment được khởi tạo bằng 0, AdamW thực hiện hiệu chỉnh bias trước khi cập nhật tham số:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}} - \eta \lambda \theta_t$$

Trong đó η là learning rate, λ là weight decay, còn ε là hằng số nhỏ tránh chia cho 0. AdamW tách weight decay khỏi gradient thích nghi nên thường phù hợp với Transformer. Notebook dùng learning rate 10^{-4} và weight decay 10^{-4} cho mô hình chính. ViT-B/16 pretrained đạt trên 94% ngay epoch đầu, trong khi ViT-B/32 không có pretrained weight và phải huấn luyện từ đầu chỉ đạt 56,38%; điều này cho thấy trạng thái pretrained phải được xét khi so sánh hai mô hình.

2.8 Chỉ số đánh giá và khả năng tổng quát

Accuracy là tỷ lệ số mẫu được dự đoán đúng trên tổng số mẫu đánh giá:

$$\text{Accuracy} = \frac{1}{M} \sum_{i=1}^M \mathbb{1}(\hat{y}_i = y_i) \times 100\%$$

Hàm chỉ thị bằng 1 khi nhãn dự đoán trùng nhãn thật và bằng 0 khi dự đoán sai. Với 10.000 ảnh test, best accuracy 95,32% tương ứng 9.532 ảnh đúng và 468 ảnh sai. Tuy nhiên, chỉ theo dõi accuracy là chưa đủ; cần quan sát đồng thời train loss, test loss và khoảng cách tổng quát:

$$\text{Generalization gap} = \text{Accuracy}_{\text{train}} - \text{Accuracy}_{\text{test}}$$

Khoảng cách giữa train accuracy và test accuracy tăng cho thấy mô hình có nguy cơ overfitting. Trong notebook, epoch 5 đạt train accuracy 98,04% nhưng test accuracy giảm còn 94,38%, thấp hơn mức 95,32% ở epoch 4. Vì vậy checkpoint tốt nhất được chọn theo kết quả trên tập test/validation thay vì chỉ lấy epoch cuối. Việc phân tích thêm các ảnh dự đoán sai giúp xác định các lớp dễ nhầm, ảnh mờ, vật thể nhỏ hoặc nền phức tạp mà accuracy tổng quát không thể hiện rõ.

2.9 So sánh CNN với ViT và ảnh hưởng của siêu tham số

CNN khai thác đặc trưng ảnh bằng các kernel tích chập cục bộ. Với đầu vào X và kernel W , giá trị đầu ra tại vị trí (i,j) , kênh o có thể biểu diễn:

$$Y_{i,j,o} = b_o + \sum_c \sum_u \sum_v W_{u,v,c,o} X_{i+u,j+v,c}$$

Nhờ chia sẻ trọng số và receptive field cục bộ, CNN có inductive bias phù hợp với ảnh và thường học tốt khi dữ liệu không quá lớn. ViT ít giả định hơn về cấu trúc ảnh, dùng self-attention để học quan hệ toàn cục nhưng thường phụ thuộc nhiều vào

pretraining và data augmentation. Vì vậy, so sánh CNN với ViT cần kiểm soát số epoch, optimizer, trạng thái pretrained và ngân sách tính toán.

Learning rate quyết định độ lớn bước cập nhật; quá lớn có thể làm loss dao động và phá vỡ biểu diễn pretrained, còn quá nhỏ làm hội tụ chậm. Batch size quyết định số mẫu dùng để ước lượng gradient và số bước cập nhật trong mỗi epoch:

$$\text{Số bước mỗi epoch} = \left\lceil \frac{N_{\text{train}}}{B} \right\rceil$$

Batch nhỏ tạo gradient nhiều hơn và cần nhiều bước cập nhật; batch lớn tận dụng GPU tốt nhưng yêu cầu nhiều VRAM. Trong notebook, batch size 64 vừa có thời gian huấn luyện tốt vừa đạt accuracy cao nhất trên cấu hình GPU sử dụng. Patch size cũng tác động trực tiếp đến số token theo $N=HW/P^2$: patch nhỏ tăng chi tiết nhưng làm attention tốn bộ nhớ hơn. Các siêu tham số này có quan hệ với nhau, do đó mỗi thí nghiệm chỉ nên thay đổi một yếu tố và giữ các điều kiện còn lại ổn định để kết luận có ý nghĩa.

Learning rate: điều khiển tốc độ và độ ổn định của quá trình fine-tuning.

Batch size: ảnh hưởng số bước cập nhật, độ nhiễu gradient, thời gian và mức sử dụng bộ nhớ.

Patch size: quyết định chiều dài chuỗi token, mức chi tiết của ảnh và chi phí self-attention.

Epoch và weight decay: cân bằng giữa khả năng học đủ đặc trưng và nguy cơ overfitting.

3. QUY TRÌNH THỰC HIỆN

3.1 Môi trường và dữ liệu

Notebook chạy trên Kaggle với GPU NVIDIA RTX PRO 6000 Blackwell Server Edition, VRAM 94,97 GB. Phiên bản PyTorch là 2.10.0+cu128, torchvision 0.25.0+cu128 và seed được cố định bằng 42. Dữ liệu CIFAR cùng pretrained weight được đọc từ Kaggle Input, không tải Internet trong lúc chạy.

Bảng 3: Môi trường và cấu hình thực nghiệm

Thành phần	Giá trị	Ghi chú
Nền tảng	Kaggle Notebook	Lưu kết quả tại /kaggle/working/bai7_vit_results
GPU	RTX PRO 6000 Blackwell	94,97 GB VRAM
PyTorch	2.10.0+cu128	CUDA khả dụng
Torchvision	0.25.0+cu128	CIFAR và ViT models
Seed	42	Random, NumPy và Torch
AMP	Có	Mixed precision khi dùng CUDA

3.2 Tiền xử lý và DataLoader

Ảnh train được resize về 224x224, lật ngang ngẫu nhiên, RandomCrop với padding 4, chuyển tensor và chuẩn hóa theo mean/std ImageNet. Ảnh test chỉ resize, chuyển tensor và normalize. DataLoader chính dùng batch size 64, num_workers 2 và pin_memory=True.

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(224, padding=4),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD)
])
```

3.3 Xây dựng ViT-B/16 và CNN đối chứng

Hàm `build_vit` tạo `vit_b_16` hoặc `vit_b_32`, nạp weight cục bộ nếu có và thay classification head. SimpleCNN gồm ba block Conv-BatchNorm-ReLU-MaxPool, sau đó Flatten, Linear 512, Dropout 0,3 và Linear đầu ra.

```
model = models.vit_b_16(weights=None)
model = load_local_weight(model, VIT_B16_WEIGHT)
model.heads.head = nn.Linear(model.heads.head.in_features, 10)
```

3.4 Pipeline huấn luyện và đánh giá

Mỗi epoch thực hiện `train_one_epoch`, sau đó evaluate trên test loader. Loss, accuracy và thời gian được lưu vào DataFrame/CSV. Mô hình có test accuracy tốt nhất được lưu ra file `.pth`. Optimizer chính là AdamW, loss là CrossEntropyLoss và mixed precision được bật khi CUDA khả dụng.

Bảng 4: Siêu tham số mô hình ViT-B/16 chính

Tham số	Giá trị	Vai trò
Model	ViT-B/16 pretrained	Backbone phân loại ảnh
Input size	224x224	Kích thước theo ViT torchvision
Epoch	5	Ngân sách huấn luyện
Batch size	64	Cấu hình chính
Learning rate	1e-4	AdamW
Weight decay	1e-4	Điều chuẩn trọng số
Loss	CrossEntropyLoss	Phân loại 10 lớp

3.5 Thiết kế các thí nghiệm

1. Mô hình chính ViT-B/16 trên CIFAR-10 trong 5 epoch.
2. Learning rate: 1e-4, 3e-4 và 1e-3.
3. Batch size: 16, 32 và 64.
4. Simple CNN đối chứng trên CIFAR-10.
5. ViT-B/16 với classification head 100 lớp trên CIFAR-100.
6. ViT-B/32 trên CIFAR-10 và so sánh patch size.
7. Thu thập toàn bộ ảnh dự đoán sai của best ViT-B/16.

4. KẾT QUẢ THỰC NGHIỆM

4.1 Chuẩn bị thư viện, môi trường và dữ liệu

Trước khi thực hiện Bài 1, notebook khai báo các thư viện cần thiết, lựa chọn thiết bị tính toán, cố định seed và chuẩn bị đường dẫn dữ liệu. Bước này bảo đảm các thí nghiệm phía sau sử dụng cùng cấu hình, có thể tái lập và không phải tải dữ liệu từ Internet trong quá trình chạy trên Kaggle.

Khai báo thư viện và thiết lập môi trường

Nhóm thư viện os, pathlib, shutil và time được dùng để quản lý tệp và đo thời gian; NumPy, Pandas hỗ trợ xử lý dữ liệu; Matplotlib phục vụ trực quan hóa; PyTorch và torchvision đảm nhiệm xây dựng, huấn luyện và đánh giá mô hình. Sau khi import, notebook kiểm tra CUDA, thông tin GPU và cố định seed bằng 42.

```
import os
import time
import random
import shutil
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from tqdm.auto import tqdm

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

import torchvision
import torchvision.transforms as transforms
import torchvision.models as models

print("Torch:", torch.__version__)
print("Torchvision:", torchvision.__version__)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```

    print("VRAM:", round(torch.cuda.get_device_properties(0).total_memory /
1024**3, 2), "GB")

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)

torch.backends.cudnn.benchmark = True

```

```

Torch: 2.10.0+cu128
Torchvision: 0.25.0+cu128
Device: cuda
GPU: NVIDIA RTX PRO 6000 Blackwell Server Edition
VRAM: 94.97 GB

```

Hình 1: Kết quả hiển thị kiểm tra GPU

Kết quả môi trường: Torch 2.10.0+cu128, torchvision 0.25.0+cu128, thiết bị cuda, GPU NVIDIA RTX PRO 6000 Blackwell Server Edition và VRAM 94,97 GB.

Khai báo đường dẫn dữ liệu và pretrained weight

Dữ liệu CIFAR-10, CIFAR-100 và trọng số ViT được đọc trực tiếp từ Kaggle Input. Notebook có thêm nhánh kiểm tra tên thư mục dùng dấu gạch ngang hoặc dấu gạch dưới, sau đó tạo WORK_DIR để lưu lịch sử huấn luyện, checkpoint và các kết quả trung gian.

```

CIFAR10_ROOT = Path("/kaggle/input/datasets/thinhtruong2334/cifar-10")
CIFAR100_ROOT = Path("/kaggle/input/datasets/thinhtruong2334/cifar-100")
VIT_WEIGHT_DIR = Path("/kaggle/input/datasets/thinhtruong2334/vit-weight")

if not CIFAR10_ROOT.exists():
    CIFAR10_ROOT = Path("/kaggle/input/datasets/thinhtruong2334/cifar_10")
if not CIFAR100_ROOT.exists():
    CIFAR100_ROOT = Path("/kaggle/input/datasets/thinhtruong2334/cifar_100")
if not VIT_WEIGHT_DIR.exists():
    VIT_WEIGHT_DIR = Path("/kaggle/input/datasets/thinhtruong2334/ViT_weight")

WORK_DIR = Path("/kaggle/working/bai7_vit_results")
WORK_DIR.mkdir(parents=True, exist_ok=True)

VIT_B16_WEIGHT = VIT_WEIGHT_DIR / "vit_b_16-c867db91.pth"
VIT_B32_WEIGHT = VIT_WEIGHT_DIR / "vit_b_32-dd86f899.pth"

```

```

print("CIFAR-10:", CIFAR10_ROOT.exists())
print("CIFAR-100:", CIFAR100_ROOT.exists())
print("vit_b_16 weight:", VIT_B16_WEIGHT.exists())
print("vit_b_32 weight:", VIT_B32_WEIGHT.exists())

CIFAR10_ROOT: /kaggle/input/datasets/thinhtruong2334/cifar-10
CIFAR100_ROOT: /kaggle/input/datasets/thinhtruong2334/cifar-100
VIT_WEIGHT_DIR: /kaggle/input/datasets/thinhtruong2334/vit-weight
VIT_B16_WEIGHT: /kaggle/input/datasets/thinhtruong2334/vit-weight/vit_b_16-c867db91.pth
VIT_B32_WEIGHT: /kaggle/input/datasets/thinhtruong2334/vit-weight/vit_b_32-dd86f899.pth

Kiểm tra tồn tại:
CIFAR-10: True
CIFAR-100: True
vit_b_16 weight: True
vit_b_32 weight: False

```

Hình 2: Kiểm tra dữ liệu CIFAR-10 và CIFAR_100

Kết quả kiểm tra cho thấy CIFAR-10, CIFAR-100 và trọng số ViT-B/16 đều tồn tại. Tập trọng số ViT-B/32 không có trong thư mục đầu vào, vì vậy thí nghiệm ViT-B/32 ở phần sau phải huấn luyện từ đầu.

4.1 Bài 2: Tải dataset CIFAR-10

Notebook kiểm tra đường dẫn CIFAR-10, CIFAR-100 và weight cục bộ trước khi tạo dataset. Kết quả xác nhận CIFAR-10, CIFAR-100 và weight ViT-B/16 đều tồn tại; weight ViT-B/32 không tồn tại.

```

train_dataset = torchvision.datasets.CIFAR10(
    root=str(CIFAR10_ROOT),
    train=True,
    download=False,
    transform=train_transform
)

test_dataset = torchvision.datasets.CIFAR10(
    root=str(CIFAR10_ROOT),
    train=False,
    download=False,
    transform=test_transform
)

```

Kết quả notebook:

```

Train samples: 50000
Test samples: 10000
Classes: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

```

Hình 3: Kiểm tra mẫu dataset CIFAR- 10

CIFAR-10: True

CIFAR-100: True

vit_b_16 weight: True

vit_b_32 weight: False

Train samples: 50000

Test samples: 10000

Classes: ['airplane', 'automobile', 'bird', 'cat', 'deer',
'dog', 'frog', 'horse', 'ship', 'truck']

Dataset được tải đủ 50.000 ảnh train và 10.000 ảnh test. Nhãn lớp đúng thứ tự chuẩn của torchvision, thuận lợi cho việc diễn giải inference và các trường hợp dự đoán sai.

4.2 Bài 2: Hiển thị ảnh trong dataset

Một batch train được lấy từ DataLoader, giải chuẩn hóa theo mean/std ImageNet và hiển thị 12 ảnh đầu. Các nhãn frog, airplane, deer, automobile, bird, horse, truck, ship và dog được gán trực tiếp từ classes.

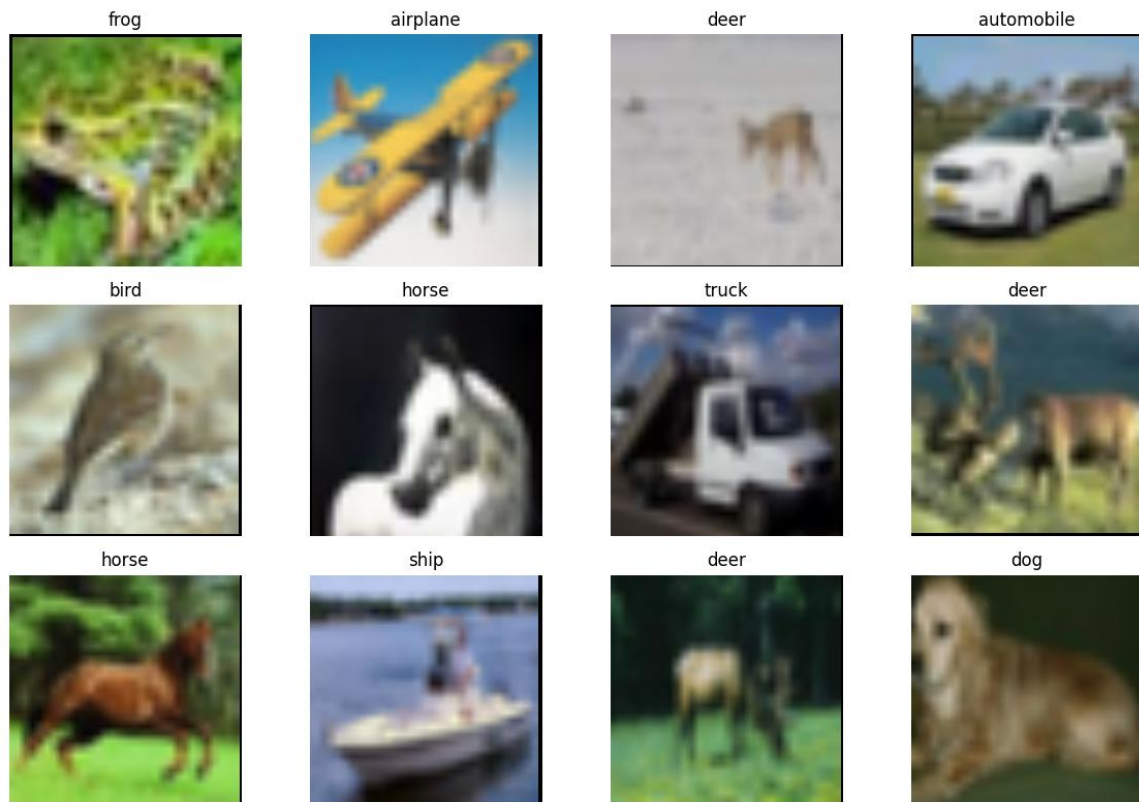
```
def denormalize(img_tensor):
    mean = torch.tensor(IMAGENET_MEAN).view(3, 1, 1)
    std = torch.tensor(IMAGENET_STD).view(3, 1, 1)
    img = img_tensor.cpu() * std + mean
    img = torch.clamp(img, 0, 1)
    return img

images, labels = next(iter(train_loader))

plt.figure(figsize=(12, 8))

for i in range(12):
    plt.subplot(3, 4, i + 1)
    img = denormalize(images[i])
    plt.imshow(img.permute(1, 2, 0))
    plt.title(classes[labels[i].item()])
    plt.axis("off")

plt.tight_layout()
plt.show()
```



Hình 4: Mười hai ảnh mẫu trong CIFAR-10 từ notebook

Ảnh CIFAR gốc chỉ 32x32 nên sau khi resize lên 224x224 vẫn khá mờ. Tuy nhiên, nhãn hiển thị phù hợp với đối tượng chính; điều này xác nhận pipeline transform, DataLoader và hàm denormalize hoạt động đúng.

4.3 Bài 3: Huấn luyện Vision Transformer trong 5 epoch

ViT-B/16 được nạp pretrained weight thành công và fine-tune toàn bộ backbone với learning rate 1e-4, batch size 64. Kết quả từng epoch được ghi lại như sau:

```
EPOCHS_MAIN = 5
LR_MAIN = 1e-4

vit_b16 = build_vit(
    model_name="vit_b_16",
    num_classes=10,
    weight_path=VIT_B16_WEIGHT,
    freeze_backbone=False
)

vit_b16, hist_vit_b16, best_acc_vit_b16 = run_experiment(
    name="vit_b16_cifar10_main",
    model=vit_b16,
    train_loader=train_loader,
    test_loader=test_loader,
    epochs=EPOCHS_MAIN,
```

```

    lr=LR_MAIN
)

hist_vit_b16

```

Epoch [5/5] | Train Loss: 0.0589 | Train Acc: 98.04% | Test Loss: 0.1780 | Test Acc: 94.38% | Time: 49.3s
 Best accuracy: 95.32 %
 Saved best model: /kaggle/working/bai7_vit_results/vit_b16_cifar10_main_best.pth

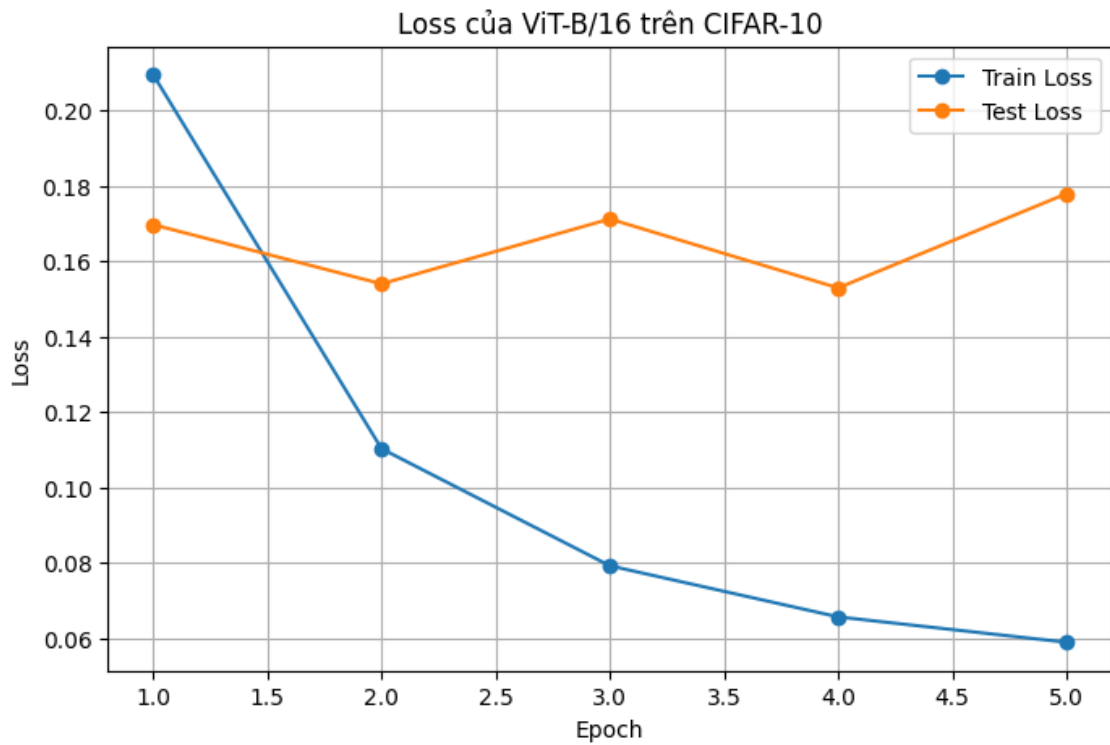
	experiment	epoch	train_loss	train_acc	test_loss	test_acc	time_sec
0	vit_b16_cifar10_main	1	0.209510	93.266	0.169734	94.35	50.753842
1	vit_b16_cifar10_main	2	0.110268	96.292	0.154019	95.11	48.824734
2	vit_b16_cifar10_main	3	0.079200	97.326	0.171249	94.96	49.025712
3	vit_b16_cifar10_main	4	0.065628	97.816	0.152936	95.32	49.312194
4	vit_b16_cifar10_main	5	0.058898	98.044	0.177957	94.38	49.277875

Hình 5: Kết quả huấn luyện Vision Transformer trong 5 epoch

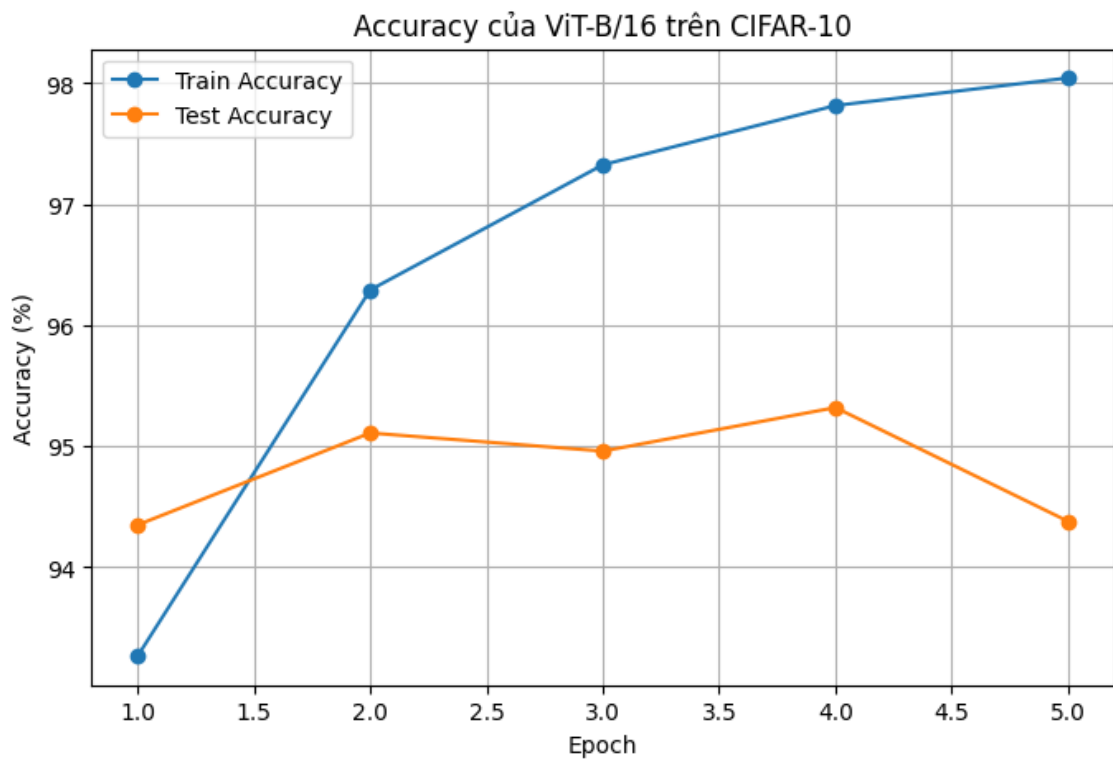
Bảng 5: Kết quả huấn luyện ViT-B/16 theo từng epoch

Epoch	Train loss	Train acc	Test loss	Test acc	Thời gian
1	0,2095	93,27%	0,1697	94,35%	50,8 s
2	0,1103	96,29%	0,1540	95,11%	48,8 s
3	0,0792	97,33%	0,1712	94,96%	49,0 s
4	0,0656	97,82%	0,1529	95,32%	49,3 s
5	0,0589	98,04%	0,1780	94,38%	49,3 s

Best test accuracy đạt 95,32% ở epoch 4. Train loss giảm đều đến 0,0589 và train accuracy tăng lên 98,04%, nhưng test loss dao động và test accuracy giảm ở epoch 5, cho thấy dấu hiệu overfitting nhẹ sau điểm tốt nhất.



Hình 6: Đường cong loss của ViT-B/16 trên CIFAR-10



Hình 7: Đường cong accuracy của ViT-B/16 trên CIFAR-10

4.4 Bài 4: Tính accuracy và inference trên tập test

Hàm evaluate chuyển mô hình sang eval(), tắt gradient, lấy argmax của logits và đếm số nhãn đúng. Best model có test accuracy 95,32%, tương đương 9.532/10.000 ảnh được phân loại đúng. Notebook còn chọn ngẫu nhiên một ảnh test để inference.

```
vit_b16.eval()

idx = random.randint(0, len(test_dataset) - 1)
image, label = test_dataset[idx]

with torch.no_grad():
    output = vit_b16(image.unsqueeze(0).to(device))
    pred = output.argmax(dim=1).item()

img_show = denormalize(image)

plt.figure(figsize=(4, 4))
plt.imshow(img_show.permute(1, 2, 0))
plt.axis("off")
plt.title(f"True: {classes[label]} | Pred: {classes[pred]}")
plt.show()

print("Nhãn thật:", classes[label])
print("Dự đoán:", classes[pred])
```



Hình 7: Kết quả inference một ảnh automobile

Kết quả notebook in Nhãn thật: automobile và Dự đoán: automobile. Ảnh khá mờ nhưng mô hình vẫn nhận diện đúng lớp xe hơi, minh họa khả năng chuyển giao đặc trưng từ pretrained ViT.

4.5 Bài 5: Thay đổi learning rate

Ba mô hình ViT-B/16 đều được khởi tạo lại từ cùng pretrained weight và huấn luyện 5 epoch. Vì các yếu tố khác giữ nguyên, chênh lệch kết quả phản ánh trực tiếp ảnh hưởng của learning rate.

```
LR_EPOCHS = 5
LR_VALUES = [1e-4, 3e-4, 1e-3]

lr_results = []

for lr in LR_VALUES:
    print("\nRunning learning rate:", lr)

    model_lr = build_vit(
        model_name="vit_b_16",
        num_classes=10,
        weight_path=VIT_B16_WEIGHT,
        freeze_backbone=False
    )

    _, hist_lr, best_acc = run_experiment(
        name=f"vit_b16_lr_{lr}",
        model=model_lr,
        train_loader=train_loader,
        test_loader=test_loader,
        epochs=LR_EPOCHS,
        lr=lr
    )
```

Kết quả:

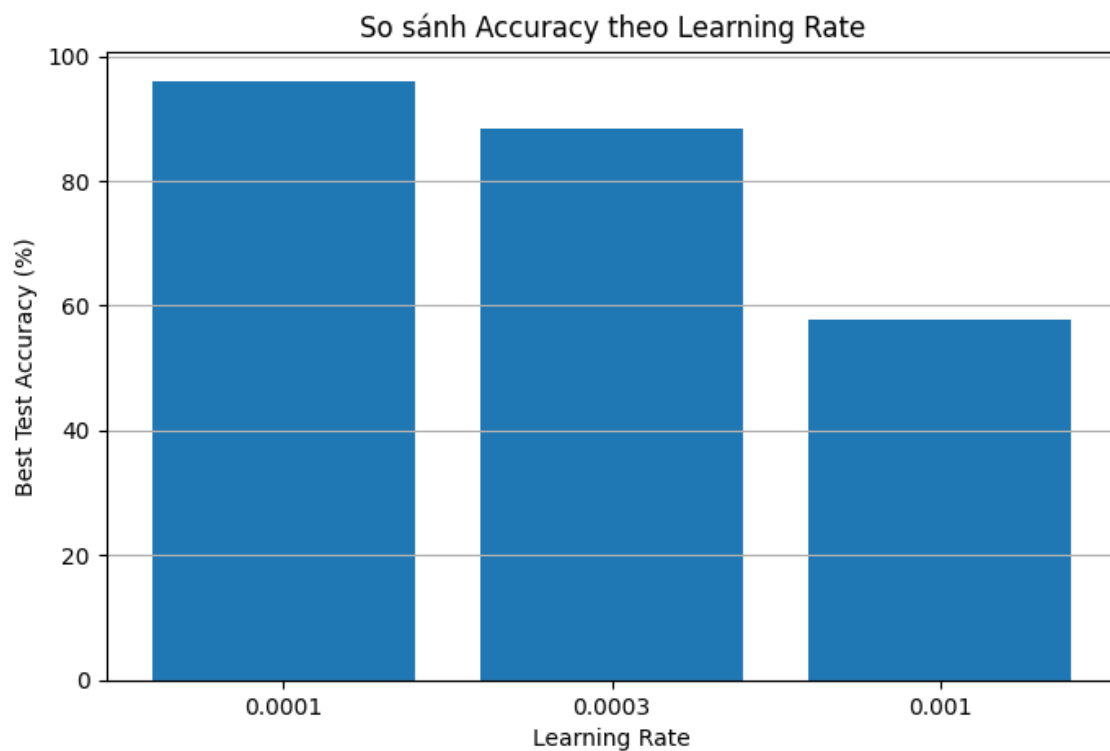
Epoch [5/5] | Train Loss: 1.1573 | Train Acc: 58.23% | Test Loss: 1.1444 | Test Acc: 57.86% | Time: 49.4s
Best accuracy: 57.86 %
Saved best model: /kaggle/working/bai7_vit_results/vit_b16_lr_0.001_best.pth

	learning_rate	best_test_acc	final_test_acc	final_train_loss	final_test_loss
0	0.0001	95.94	95.94	0.056992	0.129494
1	0.0003	88.31	88.31	0.253573	0.337663
2	0.0010	57.86	57.86	1.157343	1.144426

Hình 8: Kết quả thay đổi learning rate

Bảng 6: Kết quả thử nghiệm learning rate

Learning rate	Best test acc	Final test acc	Final train loss	Final test loss
0,0001	95,94%	95,94%	0,056992	0,129494
0,0003	88,31%	88,31%	0,253573	0,337663
0,0010	57,86%	57,86%	1,157343	1,144426



Hình 9: So sánh accuracy theo learning rate

Learning rate $1e-4$ tốt nhất với 95,94%. Tăng lên $3e-4$ làm accuracy giảm 7,63 điểm phần trăm; $1e-3$ chỉ đạt 57,86%. Với fine-tuning pretrained transformer, bước cập nhật quá lớn dễ phá vỡ biểu diễn đã học và làm tối ưu hóa thiếu ổn định.

4.6 Bài 6: Thử các batch size 16, 32, 64

Notebook giữ learning rate $1e-4$ và thử batch size 16, 32, 64. Mỗi cấu hình dùng lại pretrained ViT-B/16 và chạy 5 epoch

```
BATCH_EPOCHS = 5
BATCH_VALUES = [16, 32, 64]

batch_results = []
```

```

for bs in BATCH_VALUES:
    print("\nRunning batch size:", bs)

    train_loader_bs = DataLoader(
        train_dataset,
        batch_size=bs,
        shuffle=True,
        num_workers=NUM_WORKERS,
        pin_memory=True
    )

    test_loader_bs = DataLoader(
        test_dataset,
        batch_size=bs,
        shuffle=False,
        num_workers=NUM_WORKERS,
        pin_memory=True
    )

```

Kết quả:

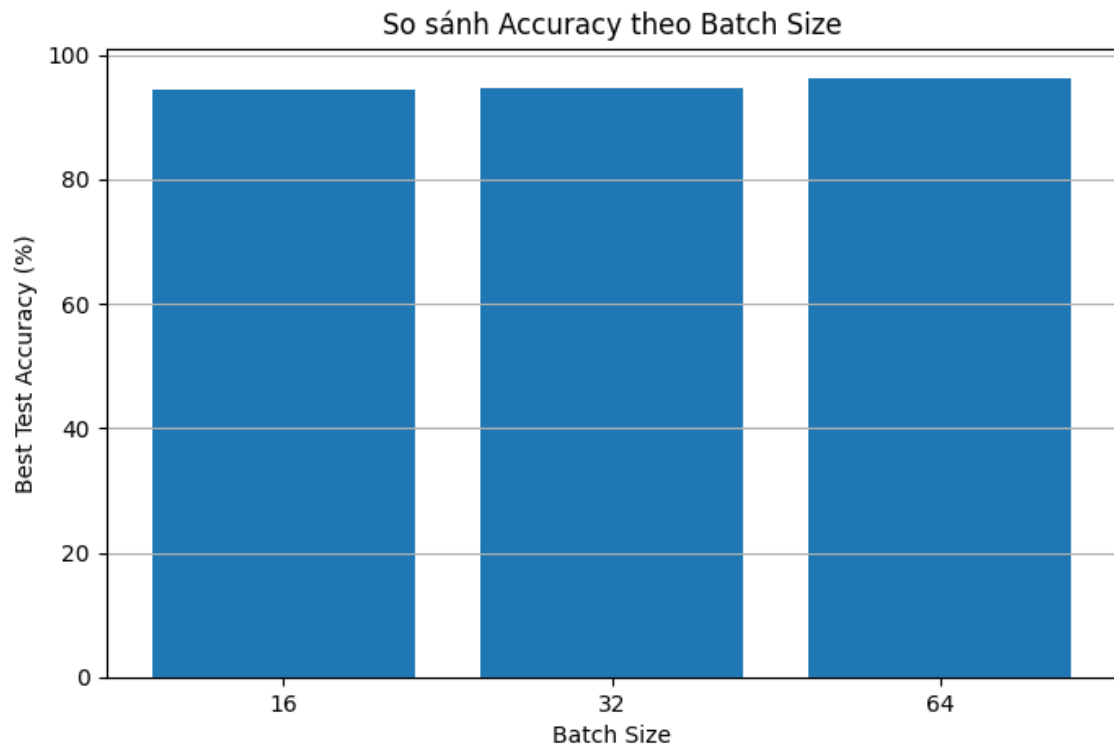
Epoch [5/5] | Train Loss: 0.0615 | Train Acc: 97.92% | Test Loss: 0.1306 | Test Acc: 96.01% | Time: 49.3s
 Best accuracy: 96.22 %
 Saved best model: /kaggle/working/bai7_vit_results/vit_b16_batch_64_best.pth

	batch_size	best_test_acc	final_test_acc	avg_time_per_epoch_sec	final_train_loss	final_test_loss
0	16	94.49	94.49	73.502981	0.111857	0.161415
1	32	94.61	94.61	56.278292	0.084111	0.180925
2	64	96.22	96.01	49.450591	0.061506	0.130625

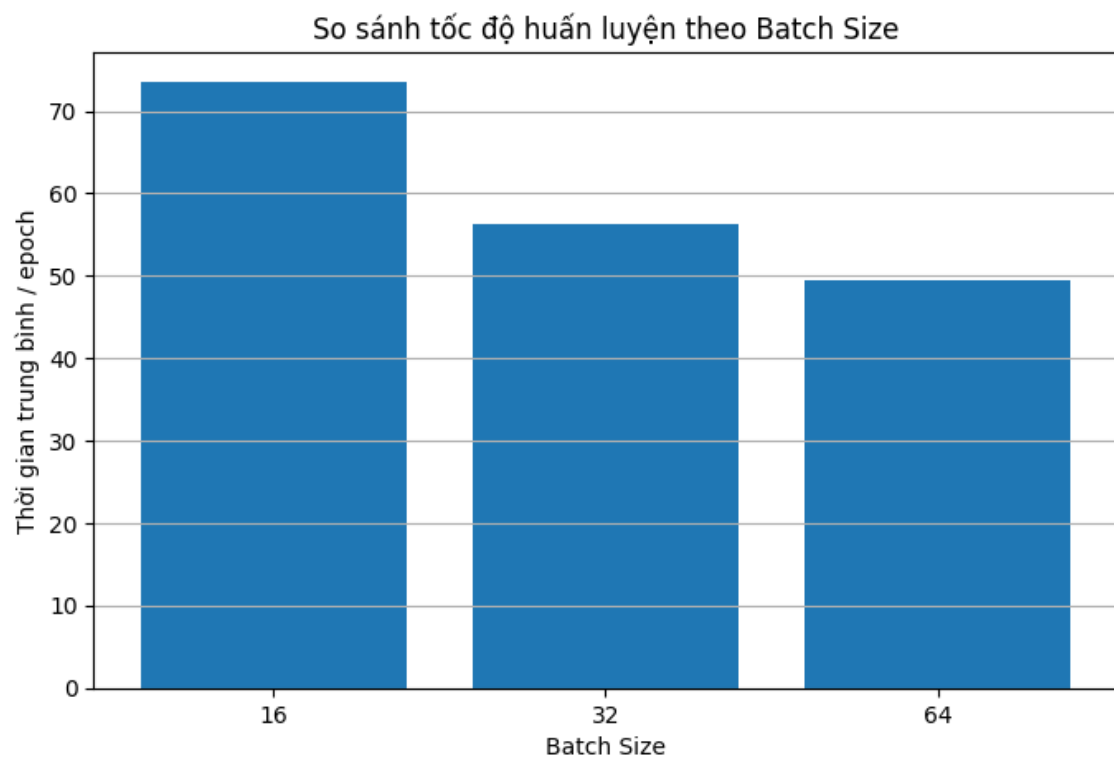
Hình 10: So sánh accuracy theo các batchsize 16, 32, 64

Bảng 7: Kết quả thử nghiệm batch size

Batch size	Best test acc	Final test acc	Thời gian TB/epoch	Final test loss
16	94,49%	94,49%	73,50 s	0,161415
32	94,61%	94,61%	56,28 s	0,180925
64	96,22%	96,01%	49,45 s	0,130625



Hình 11: So sánh accuracy theo batch size



Hình 12: So sánh thời gian huấn luyện theo batch size

Batch size 64 vừa đạt best accuracy cao nhất 96,22% vừa có thời gian trung bình thấp nhất 49,45 giây/epoch. GPU có VRAM lớn nên batch 64 tận dụng song song tốt

hơn. Trong phạm vi ba giá trị này chưa thấy bất lợi của batch lớn, nhưng không thể suy rộng cho batch lớn hơn nhiều.

4.7 Bài 7: So sánh Vision Transformer và CNN

SimpleCNN được huấn luyện 5 epoch với AdamW learning rate $1e-3$. Kết quả được so sánh với ViT-B/16 chính.

```
CNN_EPOCHS = 5
CNN_LR = 1e-3

simple_cnn = SimpleCNN(num_classes=10)

simple_cnn, hist_cnn, best_acc_cnn = run_experiment(
    name="simple_cnn_cifar10",
    model=simple_cnn,
    train_loader=train_loader,
    test_loader=test_loader,
    epochs=CNN_EPOCHS,
    lr=CNN_LR
)

hist_cnn
```

Kết quả:

Epoch [5/5] | Train Loss: 1.8074 | Train Acc: 29.31% | Test Loss: 1.5952 | Test Acc: 40.14% | Time: 18.6s
Best accuracy: 40.14 %
Saved best model: /kaggle/working/bai7_vit_results/simple_cnn_cifar10_best.pth

	experiment	epoch	train_loss	train_acc	test_loss	test_acc	time_sec
0	simple_cnn_cifar10	1	3.047293	21.968	1.865223	30.28	18.130616
1	simple_cnn_cifar10	2	1.944035	26.030	1.737455	36.07	18.134899
2	simple_cnn_cifar10	3	1.894053	26.532	1.668692	37.19	18.298628
3	simple_cnn_cifar10	4	1.851634	28.156	1.663734	34.79	18.259305
4	simple_cnn_cifar10	5	1.807450	29.310	1.595185	40.14	18.563748

	model	epochs	best_test_acc	final_test_acc	final_test_loss
0	ViT-B/16	5	95.32	94.38	0.177957
1	Simple CNN	5	40.14	40.14	1.595185

Hình 13: Kết quả so sánh Vision Transformer và CNN cơ bản

Bảng 8: So sánh ViT-B/16 và Simple CNN

Mô hình	Epoch	Best test acc	Final test acc	Final test loss
ViT-B/16 pretrained	5	95,32%	94,38%	0,177957
Simple CNN	5	40,14%	40,14%	1,595185

ViT cao hơn CNN 55,18 điểm phần trăm. Tuy nhiên, ViT dùng pretrained weight còn CNN huấn luyện từ đầu, nên đây là so sánh pipeline thực tế chứ chưa phải thí nghiệm cô lập hoàn toàn khác biệt Transformer-CNN. CNN cũng có lớp fully connected rất lớn sau ảnh 224x224, làm tối ưu hóa khó trong 5 epoch.

4.8 Bài 8: Thử nghiệm dataset CIFAR-100

CIFAR-100 được tải thành công với 50.000 ảnh train, 10.000 ảnh test và 100 lớp. Classification head của ViT-B/16 được thay thành 100 đầu ra, sau đó fine-tune 5 epoch với learning rate 1e-4, batch size 64.

```
cifar100_train_dataset = torchvision.datasets.CIFAR100(  
    root=str(CIFAR100_ROOT),  
    train=True,  
    download=False,  
    transform=train_transform  
)  
  
cifar100_test_dataset = torchvision.datasets.CIFAR100(  
    root=str(CIFAR100_ROOT),  
    train=False,  
    download=False,  
    transform=test_transform  
)
```

Kết quả:

Epoch [5/5] | Train Loss: 0.1893 | Train Acc: 93.96% | Test Loss: 0.6009 | Test Acc: 84.01% | Time: 49.5s
 Best accuracy: 84.01 %
 Saved best model: /kaggle/working/bai7_vit_results/vit_b16_cifar100_best.pth

	experiment	epoch	train_loss	train_acc	test_loss	test_acc	time_sec
0	vit_b16_cifar100	1	1.000190	74.324	0.685301	80.36	49.277869
1	vit_b16_cifar100	2	0.442006	86.492	0.612135	82.12	49.408004
2	vit_b16_cifar100	3	0.303134	90.620	0.585321	83.59	50.012706
3	vit_b16_cifar100	4	0.234996	92.598	0.653492	82.31	50.175866
4	vit_b16_cifar100	5	0.189300	93.960	0.600860	84.01	49.481822

Hình 14: Kết quả thử nghiệm dataset CIFAR-100

Bảng 9: Kết quả ViT-B/16 trên CIFAR-100

Epoch	Train loss	Train acc	Test loss	Test acc
1	1,0002	74,32%	0,6853	80,36%
2	0,4420	86,49%	0,6121	82,12%
3	0,3031	90,62%	0,5853	83,59%
4	0,2350	92,60%	0,6535	82,31%
5	0,1893	93,96%	0,6009	84,01%

Best accuracy đạt 84,01% ở epoch 5, thấp hơn CIFAR-10 11,31 điểm. Nguyên nhân chính là số lớp tăng gấp 10, số mẫu mỗi lớp giảm và nhiều lớp CIFAR-100 có đặc trưng gần nhau hơn.

4.9 Bài 9: Thử mô hình ViT-B/32

Notebook không tìm thấy file vit_b_32-dd86f899.pth, vì vậy ViT-B/32 được huấn luyện từ đầu. Mô hình vẫn chạy đủ 5 epoch với patch size 32 nhưng kết quả không thể dùng để đánh giá riêng ảnh hưởng patch size so với ViT-B/16 pretrained.

```
VIT_B32_EPOCHS = 5
VIT_B32_LR = 1e-4

vit_b32 = build_vit(
    model_name="vit_b_32",
    num_classes=10,
    weight_path=VIT_B32_WEIGHT,
    freeze_backbone=False
)

vit_b32, hist_vit_b32, best_acc_vit_b32 = run_experiment(
```

```

name="vit_b32_cifar10",
model=vit_b32,
train_loader=train_loader,
test_loader=test_loader,
epochs=VIT_B32_EPOCHS,
lr=VIT_B32_LR
)

hist_vit_b32

```

Kết quả notebook:

Epoch [5/5] | Train Loss: 1.1209 | Train Acc: 59.45% | Test Loss: 1.1998 | Test Acc: 56.38% | Time: 21.6s
 Best accuracy: 56.38 %
 Saved best model: /kaggle/working/bai7_vit_results/vit_b32_cifar10_best.pth

	experiment	epoch	train_loss	train_acc	test_loss	test_acc	time_sec
0	vit_b32_cifar10	1	1.690070	37.600	1.528841	43.58	21.250033
1	vit_b32_cifar10	2	1.376151	49.920	1.407519	49.51	20.159151
2	vit_b32_cifar10	3	1.253277	54.644	1.317672	51.91	20.295204
3	vit_b32_cifar10	4	1.178659	57.366	1.243347	55.46	20.068813
4	vit_b32_cifar10	5	1.120915	59.452	1.199824	56.38	21.638325

Hình 15: Kết quả thử nghiệm với mô hình ViT-B/32

Không tìm thấy weight: .../vit_b_32-dd86f899.pth

Mô hình sẽ train từ đầu.

Best accuracy: 56.38 %

```

compare_vit_df = pd.DataFrame([
    {
        "model": "ViT-B/16",
        "patch_size": 16,
        "epochs": EPOCHS_MAIN,
        "best_test_acc": best_acc_vit_b16,
        "final_test_acc": hist_vit_b16["test_acc"].iloc[-1],
        "final_test_loss": hist_vit_b16["test_loss"].iloc[-1]
    },
    {
        "model": "ViT-B/32",
        "patch_size": 32,
        "epochs": VIT_B32_EPOCHS,
        "best_test_acc": best_acc_vit_b32,
        "final_test_acc": hist_vit_b32["test_acc"].iloc[-1],
        "final_test_loss": hist_vit_b32["test_loss"].iloc[-1]
    }
])

```


	model	patch_size	epochs	best_test_acc	final_test_acc	final_test_loss
0	ViT-B/16	16	5	95.32	94.38	0.177957
1	ViT-B/32	32	5	56.38	56.38	1.199824

 Generate
  Code

Bảng 10: So sánh ViT-B/16 và ViT-B/32

Mô hình	Patch	Pretrained	Best test acc	Final test loss
ViT-B/16	16	Có	95,32%	0,177957
ViT-B/32	32	Không	56,38%	1,199824

ViT-B/16 có 196 patch nên giữ nhiều chi tiết hơn ViT-B/32 với 49 patch. Dù vậy, chênh lệch 38,94 điểm phần trăm trong notebook bị chi phối mạnh bởi pretrained status; muốn so sánh công bằng cần cả hai cùng pretrained hoặc cùng train từ đầu.

4.10 Bài 10: Phân tích các trường hợp dự đoán sai

Best ViT-B/16 được load lại và chạy trên toàn bộ 10.000 ảnh test. Notebook ghi nhận 468 ảnh sai, tương ứng accuracy 95,32%, khớp kết quả best model. Mười hai lỗi đầu tiên được trực quan hóa.

Kết quả:

Tổng số mẫu test: 10000
 Số mẫu dự đoán sai: 468
 Accuracy tính từ mẫu sai: 95.32 %

	idx	true_label_id	pred_label_id	true_label	pred_label
0	37	1	9	automobile	truck
1	47	9	8	truck	ship
2	57	7	3	horse	cat
3	58	4	3	deer	cat
4	59	6	3	frog	cat
5	61	3	5	cat	dog
6	68	3	7	cat	horse
7	118	2	3	bird	cat
8	147	2	5	bird	dog
9	162	6	5	frog	dog
10	183	2	4	bird	deer
11	218	8	3	ship	cat
12	224	3	4	cat	deer
13	226	6	4	frog	deer
14	232	5	7	dog	horse
15	238	5	3	dog	cat
16	275	5	3	dog	cat
17	312	8	0	ship	airplane
18	352	0	8	airplane	ship
19	355	7	4	horse	deer

Hình 16: Các trường hợp dự đoán sai

Bảng 11: Một số cặp nhãn thật - nhãn dự đoán sai

Index	Nhãn thật	Dự đoán	Nhóm nhầm lẫn
37	automobile	truck	Phương tiện đường bộ
47	truck	ship	Hình khối/nền xanh
57	horse	cat	Động vật, ảnh mờ
58	deer	cat	Động vật, góc chụp khó
59	frog	cat	Vật thể nhỏ, nền phức tạp
61	cat	dog	Hai lớp có hình dạng gần nhau

Index	Nhãn thật	Dự đoán	Nhóm nhầm lẫn
68	cat	horse	Ảnh mờ, tư thế bất thường
147	bird	dog	Chi tiết nhỏ bị mất khi resize

Show những trường hợp dự đoán sai:

```

num_show = min(12, len(wrong_samples))

plt.figure(figsize=(14, 9))

for i in range(num_show):
    sample = wrong_samples[i]
    idx = sample["idx"]

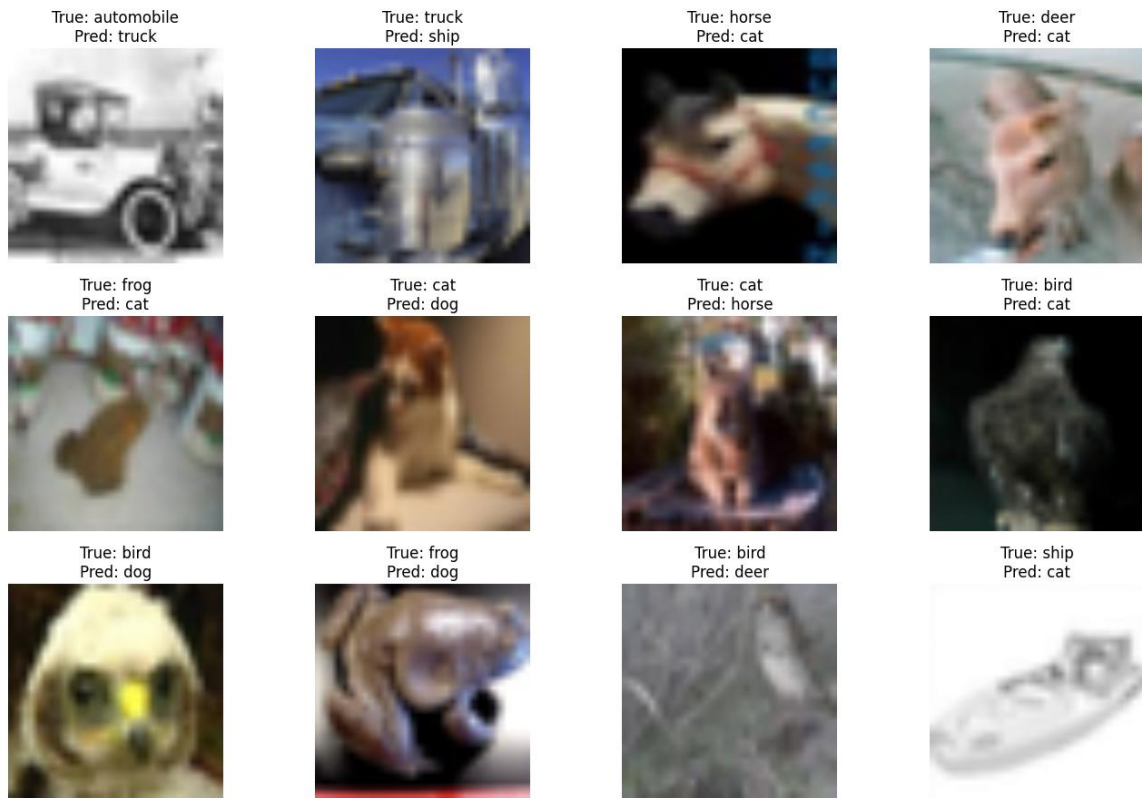
    image, label = test_dataset[idx]
    pred = sample["pred_label_id"]

    img_show = denormalize(image)

    plt.subplot(3, 4, i + 1)
    plt.imshow(img_show.permute(1, 2, 0))
    plt.title(f"True: {classes[label]}\nPred: {classes[pred]}")
    plt.axis("off")

plt.tight_layout()
plt.show()

```



Hình 17: Mười hai trường hợp ViT-B/16 dự đoán sai

Lỗi tập trung ở các lớp có đặc trưng gần nhau như cat/dog, automobile/truck, deer/horse và các ảnh quá mờ hoặc vật thể chiếm diện tích nhỏ. Resize từ 32x32 lên 224x224 không tạo thêm chi tiết, nên self-attention vẫn khó phân biệt khi tín hiệu gốc yếu.

	noi_dung	ket_qua	ghi_chu
0	ViT-B/16 trên CIFAR-10	Best accuracy = 95.32%	Mô hình chính, train 5 epoch
1	Simple CNN trên CIFAR-10	Best accuracy = 40.14%	So sánh CNN và Vision Transformer
2	ViT-B/16 trên CIFAR-100	Best accuracy = 84.01%	Thay output head thành 100 lớp
3	ViT-B/32 trên CIFAR-10	Best accuracy = 56.38%	So sánh patch size 32 với patch size 16

[Generate](#) [+ Code](#) [+ Mark](#)

Hình 18: Tổng hợp các kết quả xử dụng trong mô hình

Bảng 12: Tổng hợp kết quả chính của Lab 7

Thí nghiệm	Kết quả tốt nhất	Ghi chú
ViT-B/16 - CIFAR-10	95,32%	Mô hình chính, 5 epoch
Learning rate	95,94% tại 1e-4	Tốt hơn 3e-4 và 1e-3
Batch size	96,22% tại batch	49,45 giây/epoch

Thí nghiệm	Kết quả tốt nhất	Ghi chú
	64	
Simple CNN - CIFAR-10	40,14%	Train từ đầu
ViT-B/16 - CIFAR-100	84,01%	Head 100 lớp
ViT-B/32 - CIFAR-10	56,38%	Thiếu pretrained weight
Dự đoán sai	468/10.000	Accuracy 95,32%

5. PHÂN TÍCH VÀ THẢO LUẬN

5.1 Đánh giá ViT-B/16 trên CIFAR-10

ViT-B/16 đạt độ chính xác rất cao ngay từ epoch 1 (94,35%) nhờ pretrained weight. Accuracy tăng đến 95,32% ở epoch 4 trong khi train accuracy đạt 97,82%. Đến epoch 5, train accuracy tiếp tục tăng nhưng test accuracy giảm còn 94,38% và test loss tăng lên 0,1780, cho thấy mô hình bắt đầu khớp mạnh hơn với train set. Lưu best checkpoint là quyết định đúng để tránh dùng epoch cuối kém hơn.

5.2 Ảnh hưởng của learning rate và batch size

Learning rate là yếu tố nhạy nhất: $1e-4$ đạt 95,94%, $3e-4$ đạt 88,31%, còn $1e-3$ chỉ 57,86%. Fine-tuning transformer cần bước cập nhật nhỏ để không làm hỏng biểu diễn pretrained. Batch size 64 phù hợp nhất với GPU lớn, cho cả accuracy tốt nhất và thời gian thấp nhất. Tuy nhiên, kết luận batch size còn phụ thuộc phần cứng, learning rate và số epoch.

5.3 So sánh ViT và CNN

ViT-B/16 vượt xa SimpleCNN trong notebook vì kết hợp backbone lớn, pretrained weight và self-attention toàn cục. SimpleCNN chỉ có ba block convolution và được train từ đầu trong 5 epoch. Do điều kiện khởi tạo không đồng nhất, thí nghiệm cho thấy lợi ích của transfer learning trong pipeline ViT hơn là chứng minh mọi ViT luôn tốt hơn mọi CNN. So sánh công bằng hơn cần dùng ResNet pretrained hoặc train cả hai từ đầu với ngân sách tương đương.

5.4 CIFAR-100 và ảnh hưởng số lớp

Trên CIFAR-100, accuracy giảm còn 84,01% nhưng vẫn cao so với ngân sách 5 epoch. ViT pretrained chuyển giao tốt sang head 100 lớp. Test accuracy dao động ở epoch 4 rồi tăng lại ở epoch 5, cho thấy bài toán khó hơn và cần scheduler, validation set hoặc thêm epoch để xác định điểm hội tụ ổn định.

5.5 So sánh patch size 16 và 32

Patch size 16 giữ nhiều thông tin cục bộ hơn, phù hợp với vật thể nhỏ trong CIFAR. Patch size 32 giảm chuỗi token từ 197 xuống 50, giúp epoch nhanh khoảng

20-22 giây, nhưng có thể mất chi tiết. Tuy nhiên, ViT-B/32 không có pretrained weight nên kết quả 56,38% không thể quy hoàn toàn cho patch size. Đây là giới hạn quan trọng của thí nghiệm.

5.6 Phân tích lỗi dự đoán

Các lớp động vật có hình dạng và texture gần nhau tạo nhiều lỗi cat/dog, deer/horse, frog/cat. Nhóm phương tiện nhằm automobile/truck hoặc ship/airplane khi góc nhìn, nền và silhouette tương tự. Ảnh 32x32 chứa ít pixel, nên blur và vật thể nhỏ làm mô hình thiếu bằng chứng thị giác dù ảnh được phóng lên 224x224.

5.7 Lỗi thực nghiệm, hạn chế và hướng cải thiện

Bảng 13: Lỗi/cảnh báo thực nghiệm và hướng khắc phục

Hiện tượng	Nguyên nhân	Khắc phục
AssertionError: can only test a child process	DataLoader multiprocessing giải phóng worker trong môi trường notebook	Đặt num_workers=0/1, hạn chế tạo lại loader, thử persistent_workers=False
Không tìm thấy weight ViT-B/32	File weight không có trong Kaggle Input	Bổ sung đúng weight hoặc so sánh cả hai mô hình từ đầu
Cảnh báo torch.cuda.amp cũ	API GradScaler/autocast thay đổi	Dùng torch.amp.GradScaler('cuda') và torch.autocast('cuda')
Test loss dao động sau epoch 2	Overfitting nhẹ và learning rate cố định	Early stopping, scheduler, validation set và augmentation mạnh hơn
So sánh ViT-CNN chưa công bằng	ViT pretrained, CNN train từ đầu và kiến trúc khác quy mô	Dùng CNN pretrained hoặc cân bằng tham số/ngân sách

Thêm learning-rate scheduler và early stopping theo validation loss.

Thử MixUp, CutMix, RandAugment và label smoothing cho ảnh CIFAR nhỏ.

Bổ sung pretrained weight ViT-B/32 để cô lập ảnh hưởng patch size.

So sánh với ResNet-18/50 pretrained thay cho CNN train từ đầu.

Lập confusion matrix và tính accuracy theo từng lớp để định lượng nhóm lỗi.

Thử mô hình phù hợp ảnh nhỏ hoặc patch size nhỏ hơn, tránh phụ thuộc resize 32 lên 224.

6. KẾT LUẬN

Bài thực hành đã triển khai đầy đủ pipeline Vision Transformer trên CIFAR: chuẩn bị dữ liệu, augmentation, fine-tune ViT-B/16, đánh giá accuracy, inference, thay đổi learning rate, batch size, so sánh CNN, thử CIFAR-100, ViT-B/32 và phân tích ảnh sai. ViT-B/16 chính đạt best accuracy 95,32%; cấu hình learning rate $1e-4$ đạt 95,94% và batch size 64 đạt cao nhất 96,22%.

Kết quả khẳng định pretrained weight và siêu tham số phù hợp có ảnh hưởng quyết định đến ViT. CIFAR-100 khó hơn nhưng mô hình vẫn đạt 84,01%. ViT-B/32 chỉ đạt 56,38% do thiếu pretrained weight, vì vậy cần thận trọng khi diễn giải ảnh hưởng patch size. Phân tích 468 ảnh sai cho thấy độ phân giải thấp, blur và sự tương đồng giữa các lớp là nguyên nhân chính.

Qua thực nghiệm, mô hình tốt không chỉ phụ thuộc kiến trúc mà còn phụ thuộc dữ liệu, trạng thái pretrained, cách tiền xử lý, learning rate, batch size và quy trình chọn checkpoint. Hướng cải thiện phù hợp là so sánh công bằng hơn, dùng scheduler/early stopping, tăng augmentation và đánh giá lỗi theo từng lớp.